ZAP WEB SCAN CHALLENGE | ACTIVIDAD 4

Hallazgos detectados con OWASP ZAP

Autor: Jessica Payano

HACKER WOMEN COUNCIL
2026

1. Resumen

Este informe documenta los hallazgos de seguridad identificados en la aplicación web https://ginandjuice.shop mediante un escaneo realizado con OWASP ZAP, que combina análisis pasivo (observación del tráfico) y activo (envío de peticiones de prueba). ZAP generó 24 alertas en total; este documento detalla las 14 de riesgo alto, medio y bajo que cuentan con evidencia capturada.

Distribución de los hallazgos

Riesgo alto	Riesgo medio	Riesgo bajo	Total documentado
3	4	7	14

Principales conclusiones

- Validación insuficiente de entradas: el parámetro category del catálogo es susceptible a XSS reflejado, a CRLF Injection y muestra indicios de inyección SQL. Es el riesgo más relevante y debe atenderse primero.
- Componentes desactualizados: se utiliza AngularJS 1.7.7, con vulnerabilidades públicas conocidas.
- Cabeceras de seguridad ausentes: no se configuran CSP, X-Frame-Options, HSTS ni X-Content-Type-Options, lo que reduce las defensas del navegador y agrava otros hallazgos.
- Cookies mal configuradas: las cookies AWSALB y AWSALBCORS carecen de HttpOnly, Secure y de un SameSite adecuado.
- Control de errores deficiente: la aplicación devuelve errores 500 que revelan comportamiento interno.

Alcance y metodología

El análisis se limitó al sitio https://ginandjuice.shop y se realizó con OWASP ZAP. Las alertas pasivas se basan en la observación de las respuestas del servidor; las activas, en peticiones de prueba con payloads. Cada hallazgo se presenta con ocho campos: nombre de la alerta, nivel de riesgo, URL o componente, parámetro afectado, evidencia, descripción técnica, impacto potencial y recomendación, junto con los identificadores CWE y WASC y las referencias de ZAP.

2 Hallazgos detallados

A continuación, se presenta cada hallazgo con su ficha técnica, la evidencia obtenida en ZAP y las acciones recomendadas. Los hallazgos siguen el orden de severidad que muestra la herramienta.

HALLAZGO 01	RIESGO
Cross Site Scripting (Reflected)	**ALTO**

CWE	WASC	Confianza	Origen (ZAP)
79	**8**	**Media**	**Activa (40012)**

1. Nombre de la alerta	Cross Site Scripting (Reflected)
2. Nivel de riesgo	**Alto**
3. URL o componente	https://ginandjuice.shop/catalog?category=%0A%0D%0A%0D%3Cscript%3Ealert%281%29%3B%3C%2FscRipt%3E
4. Parámetro afectado	category
5. Evidencia	`category=<script>alert(1);</script>` ZAP envió el payload en el parámetro category de la URL y la aplicación lo devolvió sin modificar en la respuesta; la evidencia coincide exactamente con el ataque enviado, lo que confirma que el contenido se refleja sin codificar.  *Figura 1. Captura de ZAP – Cross Site Scripting (Reflected).*
6. Descripción técnica	El Cross-Site Scripting (XSS) reflejado ocurre cuando la aplicación toma datos enviados por el usuario (en este caso por la query string) y los incluye en la respuesta HTML sin validarlos ni codificarlos. El navegador de la víctima interpreta ese contenido como código propio de la página y lo ejecuta. El parámetro category del endpoint /catalog es el vector afectado. Basta con que la víctima abra un enlace manipulado para que el script se ejecute en el contexto del dominio ginandjuice.shop, con acceso a todo lo que el usuario puede ver o hacer en el sitio.
7. Impacto potencial	• Robo de cookies de sesión o tokens accesibles desde JavaScript, con posible

	secuestro de la sesión del usuario. • Redirección a sitios maliciosos o descarga de malware. • Alteración del contenido visible (defacement) o creación de formularios falsos para robar credenciales (phishing). • Ejecución de acciones en nombre del usuario autenticado. • Daño reputacional y posible incumplimiento de normativas de protección de datos.
8. Recomendación	• Aplicar codificación de salida (output encoding) según el contexto donde se imprime el dato (HTML, atributo, JavaScript, URL). • Validar la entrada con listas blancas: la categoría solo debería aceptar valores esperados del catálogo. • Usar frameworks o librerías que escapen por defecto, como OWASP ESAPI o las plantillas con auto-escape. • Implementar una política Content-Security-Policy restrictiva para limitar la ejecución de scripts. • Marcar las cookies de sesión con HttpOnly para reducir el impacto de un XSS.

HALLAZGO 02

SQL Injection

RIESGO **ALTO**

CWE	WASC	Confianza	Origen (ZAP)
89	**19**	**Baja**	**Activa (40018)**

1. Nombre de la alerta	SQL Injection
2. Nivel de riesgo	**Alto**
3. URL o componente	https://ginandjuice.shop/catalog?category=%27
4. Parámetro afectado	category
5. Evidencia	`category='  →  HTTP/1.1 500 Internal Server Error` Al enviar una comilla simple en el parámetro category, la aplicación respondió con un error 500. ZAP clasifica este hallazgo con confianza baja, por lo que se recomienda validarlo manualmente para confirmar la explotación.

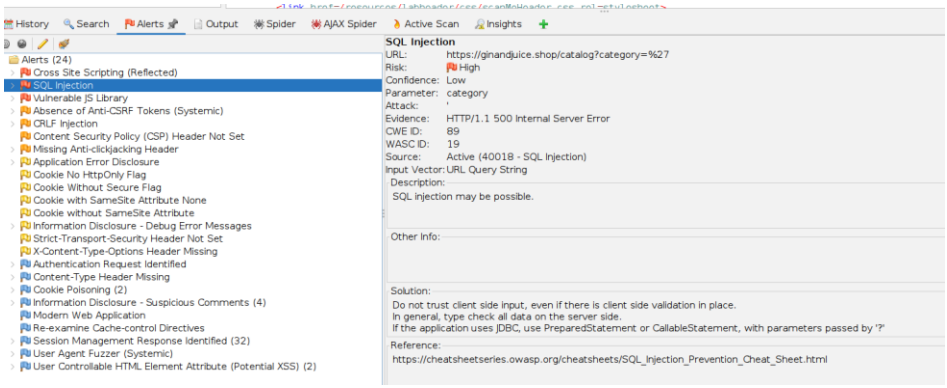


Figura 2. Captura de ZAP – SQL Injection.

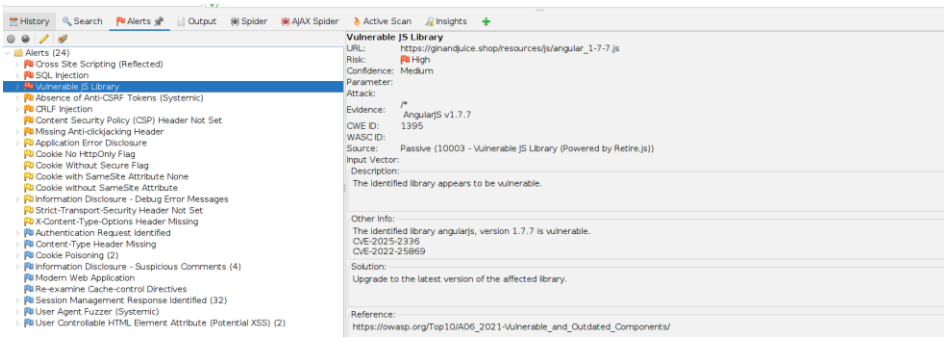
6. Descripción técnica	La inyección SQL se produce cuando la entrada del usuario se concatena directamente en una consulta a la base de datos. Un carácter especial, como la comilla simple, puede romper la sintaxis de la consulta o alterar su lógica. En este caso, la comilla enviada en el parámetro category provocó un error interno del servidor (HTTP 500), comportamiento típico de una consulta SQL mal formada. El hallazgo indica que la entrada llega al motor de base de datos sin tratamiento y que el servidor no maneja el error de forma controlada.
7. Impacto potencial	• Acceso no autorizado a información de la base de datos (usuarios, productos, datos personales). • Posible modificación o eliminación de datos y evasión de la autenticación. • En configuraciones débiles, ejecución de comandos en el servidor de base de datos. • Fuga de información técnica a través de los mensajes de error.
8. Recomendación	• Usar consultas parametrizadas (PreparedStatement o CallableStatement con parámetros '?') en todo acceso a la base de datos. • No confiar en la validación del lado del cliente: validar y verificar el tipo de todos los datos en el servidor. • Aplicar el principio de menor privilegio a la cuenta de la aplicación en la base de datos. • Manejar los errores con mensajes genéricos y registrar el detalle solo en el servidor. • Validar manualmente el hallazgo tras la corrección para confirmar que fue resuelto.

HALLAZGO 03

Vulnerable JS Library

RIESGO
ALTO

CWE	WASC	Confianza	Origen (ZAP)
1395	**-**	**Media**	**Pasiva (10003)**

1. Nombre de la alerta	Vulnerable JS Library
2. Nivel de riesgo	**Alto**
3. URL o componente	https://ginandjuice.shop/resources/js/angular_1-7-7.js
4. Parámetro afectado	N/A (componente)
5. Evidencia	`AngularJS v1.7.7 (CVE-2025-2336, CVE-2022-25869)` ZAP identificó la versión de la librería en la cabecera del archivo JavaScript y la cruzó con la base de Retire.js, que reporta vulnerabilidades públicas para la versión 1.7.7.  *Figura 3. Captura de ZAP – Vulnerable JS Library.*
6. Descripción técnica	La aplicación carga la librería AngularJS en su versión 1.7.7, que tiene vulnerabilidades conocidas y registradas, entre ellas CVE-2025-2336 y CVE-2022-25869. AngularJS es una versión antigua del framework y ya no recibe soporte oficial. Usar componentes desactualizados es una de las categorías del OWASP Top 10 (A06:2021 – Vulnerable and Outdated Components): los atacantes conocen estas fallas y pueden explotarlas sin necesidad de descubrirlas.
7. Impacto potencial	• Explotación de vulnerabilidades públicas de la librería, por ejemplo ataques XSS que evaden los mecanismos de sanitización de Angular. • Compromiso de los usuarios que visitan las páginas que cargan el componente. • Mayor superficie de ataque y dificultad para recibir parches de seguridad.
8. Recomendación	• Actualizar la librería a la última versión soportada, o migrar a un framework mantenido activamente. • Mantener un inventario de dependencias del lado cliente y servidor. • Evitar cargar scripts de versiones sin soporte.

HALLAZGO 04

Absence of Anti-CSRF Tokens

RIESGO
MEDIO

CWE	WASC	Confianza	Origen (ZAP)
352	**9**	**Baja**	**Pasiva (10202)**

1. Nombre de la alerta	Absence of Anti-CSRF Tokens
2. Nivel de riesgo	**Medio**
3. URL o componente	https://ginandjuice.shop/catalog/product?productId=5
4. Parámetro afectado	N/A (formulario stockCheckForm)
5. Evidencia	`<form id="stockCheckForm" action="/catalog/product/stock" method="POST">` ZAP encontró un formulario POST que no incluye ninguno de los nombres habituales de token anti-CSRF (csrf_token, CSRFToken, authenticity_token, etc.). El alcance es sistémico: afecta a varios formularios de la aplicación.  *Figura 4. Captura de ZAP – Absence of Anti-CSRF Tokens.*
6. Descripción técnica	El Cross-Site Request Forgery (CSRF) obliga al navegador de una víctima autenticada a enviar una petición a la aplicación sin que lo sepa. La defensa habitual es incluir en cada formulario un token impredecible, único por sesión, que el servidor valida. El formulario stockCheckForm, que envía datos por POST a /catalog/product/stock, no contiene ese token, por lo que el servidor no puede distinguir una petición legítima de una falsificada desde otro sitio. La confianza de la alerta es baja, por lo que debe confirmarse el efecto real de la acción.
7. Impacto potencial	• Ejecución de acciones no deseadas en nombre de un usuario autenticado. • Cambios en datos o configuraciones de la cuenta si el patrón se repite en formularios sensibles. • Pérdida de integridad de las operaciones realizadas desde el navegador.

8. Recomendación	• Implementar tokens anti-CSRF únicos por sesión o por petición en todos los formularios que cambian estado. • Usar librerías probadas, como OWASP CSRFGuard, o la protección integrada del framework. • Configurar el atributo SameSite=Lax o Strict en las cookies de sesión. • Verificar las cabeceras Origin y Referer como defensa adicional.

HALLAZGO 05

CRLF Injection

RIESGO **MEDIO**

CWE	WASC	Confianza	Origen (ZAP)
113	**25**	**Media**	**Activa (40003)**

1. Nombre de la alerta	CRLF Injection
2. Nivel de riesgo	**Medio**
3. URL o componente	https://ginandjuice.shop/catalog?category=any%0D%0ASet-cookie%3A+Tamper%3D15a29bb0-bddb-4a8d-9f73-13c065227c91
4. Parámetro afectado	category
5. Evidencia	`category=any\r\nSet-cookie: Tamper=15a29bb0-bddb-4a8d-9f73-13c065227c91` ZAP incluyó una secuencia de salto de línea (%0D%0A) seguida de una cabecera Set-Cookie. La misma cabecera apareció en la respuesta del servidor, lo que demuestra que se pueden inyectar cabeceras HTTP.  *Figura 5. Captura de ZAP - CRLF Injection.*
6. Descripción técnica	La inyección CRLF aprovecha que los caracteres de retorno de carro y salto de línea (CR y LF, %0D%0A) separan las cabeceras en una respuesta HTTP. Si la aplicación incluye entrada del usuario en una cabecera sin filtrar esos caracteres, un atacante puede añadir cabeceras propias o incluso dividir la respuesta. En este caso, el valor del parámetro category permitió establecer una cookie

	arbitraria en el navegador de la víctima. Combinado con otras técnicas, también puede derivar en envenenamiento de caché o en XSS.
7. Impacto potencial	• Establecimiento de cookies arbitrarias (fijación de sesión) en el navegador del usuario. • Inyección de cabeceras HTTP arbitrarias. • Envenenamiento de la caché de proxies o CDN. • Posible escalada a XSS mediante la manipulación de la respuesta.
8. Recomendación	• Filtrar o rechazar los caracteres CR (%0D) y LF (%0A) en cualquier dato que se use en cabeceras HTTP. • Validar el tipo y formato esperado del parámetro category. • Usar las funciones del framework para escribir cabeceras, que ya sanitizan estos caracteres. • Mantener actualizado el servidor web y los componentes intermedios.

HALLAZGO 06

Content Security Policy (CSP) Header Not Set

RIESGO

MEDIO

CWE	WASC	Confianza	Origen (ZAP)
693	**15**	**Alta**	**Pasiva (10038)**

1. Nombre de la alerta	Content Security Policy (CSP) Header Not Set
2. Nivel de riesgo	**Medio**
3. URL o componente	https://ginandjuice.shop/blog
4. Parámetro afectado	N/A (cabecera HTTP)
5. Evidencia	`Cabecera Content-Security-Policy ausente en la respuesta` La respuesta del recurso /blog no incluye la cabecera Content-Security-Policy. La confianza de la alerta es alta porque la ausencia de la cabecera es directamente verificable.

	 *Figura 6. Captura de ZAP - Content Security Policy (CSP) Header Not Set.*
6. Descripción técnica	Content Security Policy (CSP) es una capa de seguridad adicional que permite al sitio declarar de qué orígenes puede cargar el navegador scripts, estilos, imágenes, frames y fuentes. Ayuda a detectar y mitigar ataques como XSS e inyección de datos. Al no estar configurada, el navegador no tiene ninguna restricción sobre qué contenido puede ejecutar. Esto agrava el impacto de otros hallazgos, como el XSS reflejado, ya que un script inyectado se ejecutaría sin limitación.
7. Impacto potencial	• Mayor impacto de ataques XSS e inyección de contenido. • Posibilidad de cargar recursos desde dominios no autorizados. • Falta de una defensa en profundidad frente a scripts maliciosos.
8. Recomendación	• Configurar el servidor web, servidor de aplicaciones o balanceador para enviar la cabecera Content-Security-Policy. • Empezar con una política estricta (por ejemplo, default-src 'self') y ampliarla solo con los orígenes necesarios. • Probar primero en modo Content-Security-Policy-Report-Only para no romper funcionalidades. • Evitar 'unsafe-inline' y 'unsafe-eval' siempre que sea posible.

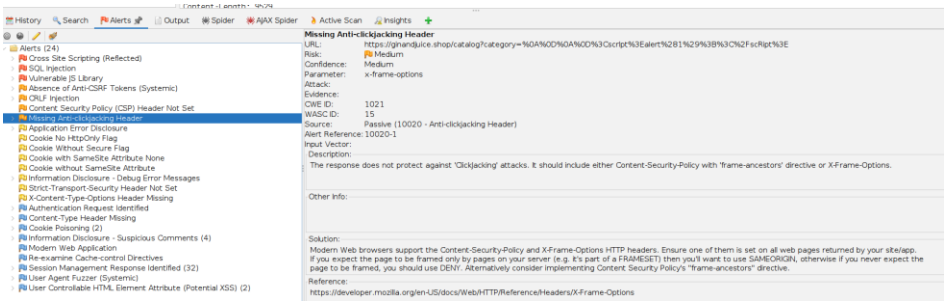
HALLAZGO 07

Missing Anti-clickjacking Header

RIESGO **MEDIO**

CWE	WASC	Confianza	Origen (ZAP)
1021	**15**	**Media**	**Pasiva (10020)**

1. Nombre de la alerta	Missing Anti-clickjacking Header
2. Nivel de riesgo	**Medio**
3. URL o componente	https://ginandjuice.shop/catalog?category=%0A%0D%0A%0D%3Cscript%3Ealert%281%29%3B%3C%2FscRipt%3E

4. Parámetro afectado	x-frame-options
5. Evidencia	`Ausencia de X-Frame-Options y de la directiva frame-ancestors de CSP` La respuesta no incluye ninguna de las dos protecciones contra el enmarcado de la página. La alerta se registró sobre la URL de /catalog que ya había sido usada para el payload XSS.  *Figura 7. Captura de ZAP – Missing Anti-clickjacking Header.*
6. Descripción técnica	El clickjacking es un ataque en el que una página maliciosa carga el sitio legítimo dentro de un frame invisible y engaña al usuario para que haga clic en botones o enlaces de la aplicación real sin darse cuenta. Para evitarlo, el servidor debe enviar la cabecera X-Frame-Options o la directiva frame-ancestors de CSP. La aplicación no incluye ninguna, por lo que cualquier sitio externo puede incrustar sus páginas.
7. Impacto potencial	• Acciones no intencionadas del usuario, como compras, cambios de configuración o aprobaciones. • Robo de interacciones con formularios o botones sensibles. • Pérdida de confianza del usuario si se usa el sitio para fraude.
8. Recomendación	• Enviar X-Frame-Options: DENY, o SAMEORIGIN si la aplicación necesita enmarcarse a sí misma. • Alternativamente, usar Content-Security-Policy con la directiva frame-ancestors. • Aplicar la cabecera en todas las páginas de la aplicación, no solo en las sensibles.

HALLAZGO 08

Application Error Disclosure

RIESGO
BAJO

CWE	WASC	Confianza	Origen (ZAP)
550	**13**	**Media**	**Pasiva (90022)**

1. Nombre de la alerta	Application Error Disclosure
2. Nivel de riesgo	**Bajo**
3. URL o componente	https://ginandjuice.shop/catalog?category=%27
4. Parámetro afectado	N/A
5. Evidencia	`HTTP/1.1 500 Internal Server Error` La respuesta a la petición con la comilla simple devolvió un error 500 sin manejo controlado. ZAP advierte que la alerta puede ser un falso positivo si el mensaje aparece en una página de documentación.  *Figura 8. Captura de ZAP – Application Error Disclosure.*
6. Descripción técnica	La aplicación devuelve mensajes de error internos al usuario cuando ocurre una excepción no controlada. Estos mensajes pueden revelar información sensible, como la ubicación de archivos, la tecnología utilizada o fragmentos de consultas. Aunque por sí sola es de riesgo bajo, esta información ayuda a un atacante a preparar otros ataques, y en este caso se relaciona con el hallazgo de SQL Injection sobre la misma URL.
7. Impacto potencial	• Revelación de detalles internos del servidor y de la aplicación. • Facilita el reconocimiento previo a ataques más complejos. • Refuerza la sospecha de otros fallos, como el de inyección SQL.
8. Recomendación	• Revisar el código de la página que genera el error y corregir la causa. • Implementar páginas de error personalizadas con mensajes genéricos. • Proporcionar al cliente solo un identificador único del error y guardar el detalle en los registros del servidor. • Desactivar los modos de depuración en producción.

HALLAZGO 09

Cookie No HttpOnly Flag

RIESGO
BAJO

CWE	WASC	Confianza	Origen (ZAP)
1004	**13**	**Media**	**Pasiva (10010)**

1. Nombre de la alerta	Cookie No HttpOnly Flag
2. Nivel de riesgo	**Bajo**
3. URL o componente	https://ginandjuice.shop/blog
4. Parámetro afectado	AWSALB
5. Evidencia	`Set-Cookie: AWSALB` La cookie AWSALB, asociada al balanceador de carga de AWS, se entrega sin el flag HttpOnly.  *Figura 9. Captura de ZAP – Cookie No HttpOnly Flag.*
6. Descripción técnica	El flag HttpOnly impide que una cookie sea leída desde JavaScript. Cuando no está presente, cualquier script que se ejecute en la página puede acceder al valor de la cookie y enviarlo a otro sitio. En este caso la cookie afectada es AWSALB, usada para la persistencia de sesión del balanceador. Si una cookie de sesión de la aplicación presentara la misma configuración, el riesgo de secuestro de sesión sería mayor.
7. Impacto potencial	• Robo de la cookie mediante un script malicioso, por ejemplo vía XSS. • Posible secuestro de sesión si la cookie identifica al usuario. • Mayor impacto de otros hallazgos de inyección de scripts.
8. Recomendación	• Asegurar que el flag HttpOnly esté activado en todas las cookies que no necesiten ser leídas desde JavaScript. • Revisar la configuración del balanceador para añadir el flag. • Verificar también las cookies propias de la aplicación.

HALLAZGO 10

Cookie Without Secure Flag

RIESGO
BAJO

CWE	WASC	Confianza	Origen (ZAP)
614	**13**	**Media**	**Pasiva (10011)**

1. Nombre de la alerta	Cookie Without Secure Flag
2. Nivel de riesgo	**Bajo**
3. URL o componente	https://ginandjuice.shop/blog
4. Parámetro afectado	AWSALB
5. Evidencia	`Set-Cookie: AWSALB` La cookie AWSALB se establece sin el atributo Secure.  *Figura 10. Captura de ZAP – Cookie Without Secure Flag.*
6. Descripción técnica	El atributo Secure indica al navegador que solo debe enviar la cookie por conexiones cifradas (HTTPS). Sin él, la cookie puede viajar en texto plano si el usuario accede por HTTP. Esto cobra relevancia junto con la ausencia de HSTS, ya que un atacante en la misma red podría forzar una conexión no cifrada y capturar la cookie.
7. Impacto potencial	• Intercepción de la cookie en redes no confiables (por ejemplo Wi-Fi públicas). • Exposición de identificadores de sesión o datos de persistencia. • Posible suplantación de la sesión del usuario.
8. Recomendación	• Activar el flag Secure en todas las cookies, especialmente las de sesión o con información sensible. • Forzar HTTPS en todo el sitio y complementar con la cabecera HSTS. • Revisar la configuración del balanceador y del servidor de aplicaciones.

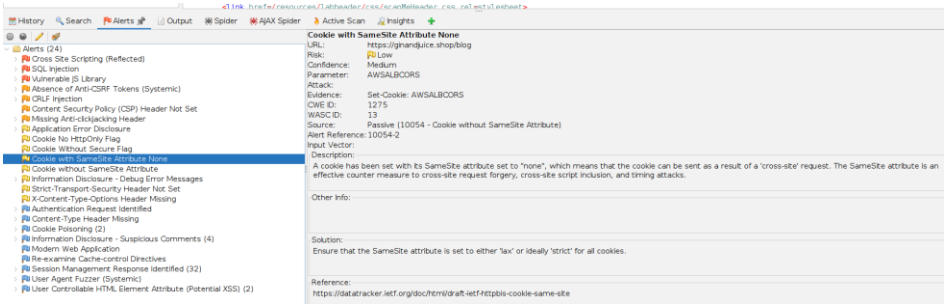
HALLAZGO 11

Cookie with SameSite Attribute None

RIESGO

BAJO

CWE	WASC	Confianza	Origen (ZAP)
1275	**13**	**Media**	**Pasiva (10054)**

1. Nombre de la alerta	Cookie with SameSite Attribute None
2. Nivel de riesgo	**Bajo**
3. URL o componente	https://ginandjuice.shop/blog
4. Parámetro afectado	AWSALBCORS
5. Evidencia	`Set-Cookie: AWSALBCORS` La cookie AWSALBCORS se establece con SameSite=None, por lo que se envía también en peticiones desde otros sitios.  *Figura 11. Captura de ZAP – Cookie with SameSite Attribute None.*
6. Descripción técnica	El atributo SameSite controla si el navegador envía una cookie en peticiones originadas desde otros sitios. El valor None permite el envío cross-site, algo necesario solo en escenarios concretos (CORS, contenido embebido). Es una contramedida eficaz contra CSRF, inclusión de scripts entre sitios y ataques de temporización; dejarla en None elimina esa protección para esta cookie.
7. Impacto potencial	• Mayor exposición a ataques CSRF. • Posible filtración de información mediante peticiones cross-site. • Reducción de las defensas del navegador frente a abusos entre sitios.
8. Recomendación	• Configurar SameSite en Lax o, idealmente, Strict en todas las cookies. • Usar None solo cuando sea indispensable, siempre junto con Secure. • Revisar con el equipo de infraestructura la configuración de las cookies del balanceador.

HALLAZGO 12

Cookie without SameSite Attribute

RIESGO

BAJO

CWE	WASC	Confianza	Origen (ZAP)
1275	**13**	**Media**	**Pasiva (10054)**

1. Nombre de la alerta	Cookie without SameSite Attribute
2. Nivel de riesgo	**Bajo**
3. URL o componente	https://ginandjuice.shop/blog
4. Parámetro afectado	AWSALB
5. Evidencia	`Set-Cookie: AWSALB` La cookie AWSALB se envía sin el atributo SameSite definido.  *Figura 12. Captura de ZAP – Cookie without SameSite Attribute.*
6. Descripción técnica	Cuando una cookie no declara el atributo SameSite, el comportamiento depende del navegador; en algunos casos se envía en peticiones cross-site. Declararlo de forma explícita es una buena práctica de seguridad. Como en el hallazgo anterior, la cookie corresponde al balanceador de carga, por lo que la corrección suele hacerse en la configuración de infraestructura.
7. Impacto potencial	• Mayor exposición a CSRF y a inclusión de scripts entre sitios. • Comportamiento inconsistente entre navegadores. • Debilitamiento de la defensa en profundidad.
8. Recomendación	• Establecer SameSite en Lax o, idealmente, Strict en todas las cookies. • Definir una política de cookies uniforme para toda la aplicación. • Verificar el resultado con un nuevo escaneo.

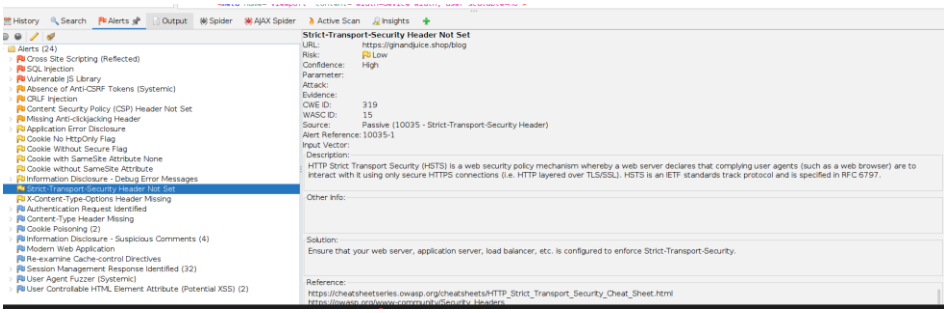
HALLAZGO 13

Strict-Transport-Security Header Not Set

RIESGO

BAJO

CWE	WASC	Confianza	Origen (ZAP)
319	**15**	**Alta**	**Pasiva (10035)**

1. Nombre de la alerta	Strict-Transport-Security Header Not Set
2. Nivel de riesgo	**Bajo**
3. URL o componente	https://ginandjuice.shop/blog
4. Parámetro afectado	N/A (cabecera HTTP)
5. Evidencia	`Cabecera Strict-Transport-Security ausente en la respuesta` La respuesta no incluye la cabecera HSTS. La confianza es alta porque la ausencia es directamente verificable.  *Figura 13. Captura de ZAP – Strict-Transport-Security Header Not Set.*
6. Descripción técnica	HTTP Strict Transport Security (HSTS) es un mecanismo por el cual el servidor indica a los navegadores que solo deben comunicarse con él mediante HTTPS (RFC 6797). Sin esta cabecera, un usuario que escriba la dirección sin https:// o que sea redirigido por un atacante puede iniciar la conexión por HTTP, abriendo la puerta a ataques de degradación (downgrade) o de intermediario (man-in-the-middle).
7. Impacto potencial	• Interceptación o modificación del tráfico si la conexión se degrada a HTTP. • Robo de cookies que viajan sin cifrar. • Menor garantía de que el usuario siempre use un canal seguro.
8. Recomendación	• Configurar el servidor web o balanceador para enviar Strict-Transport-Security con un max-age adecuado (por ejemplo 31536000). • Añadir includeSubDomains si todos los subdominios usan HTTPS. • Considerar la inclusión en la lista de precarga (preload) de los navegadores.

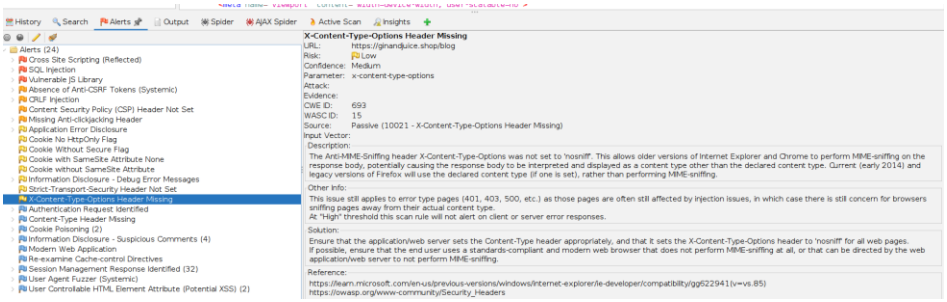
HALLAZGO 14

X-Content-Type-Options Header Missing

RIESGO

BAJO

CWE	WASC	Confianza	Origen (ZAP)
693	**15**	**Media**	**Pasiva (10021)**

1. Nombre de la alerta	X-Content-Type-Options Header Missing
2. Nivel de riesgo	**Bajo**
3. URL o componente	https://ginandjuice.shop/blog
4. Parámetro afectado	x-content-type-options
5. Evidencia	`Cabecera X-Content-Type-Options ausente (debería ser 'nosniff')` La respuesta no establece X-Content-Type-Options en 'nosniff'. ZAP indica que el problema también aplica a páginas de error (401, 403, 500).  *Figura 14. Captura de ZAP – X-Content-Type-Options Header Missing.*
6. Descripción técnica	La cabecera X-Content-Type-Options con valor 'nosniff' evita que el navegador intente adivinar el tipo de contenido de una respuesta (MIME-sniffing). Sin ella, algunos navegadores pueden interpretar un archivo como un tipo distinto al declarado. Esto puede hacer que contenido inocuo, como un texto o una imagen, sea tratado como HTML o script, y es especialmente relevante en páginas afectadas por problemas de inyección.
7. Impacto potencial	• Interpretación de contenido con un tipo distinto al declarado. • Facilita ataques XSS en combinación con otras fallas. • Debilita la política de tipos de contenido de la aplicación.
8. Recomendación	• Configurar el servidor para enviar X-Content-Type-Options: nosniff en todas las páginas, incluidas las de error. • Asegurar que la cabecera Content-Type se establezca correctamente en cada respuesta. • Recomendar a los usuarios navegadores modernos que no hagan MIME-sniffing.

3. Conclusiones y plan de remediación

El escaneo con OWASP ZAP permitió identificar debilidades en la validación de entradas, en las cabeceras de seguridad y en la configuración de las cookies. A continuación se resumen los aprendizajes del ejercicio y la tabla final de hallazgos.

¿Qué aprendí?

- ZAP entrega, para cada alerta, el nivel de riesgo, la URL, el parámetro, el payload usado, la evidencia devuelta por el servidor y los identificadores CWE/WASC. Con esos datos se puede documentar y priorizar cada hallazgo sin repetir pruebas.
- Las alertas activas (XSS, SQL Injection, CRLF) muestran el ataque enviado y la respuesta que lo confirma, mientras que las pasivas (cabeceras, cookies, librerías) se detectan solo observando las respuestas del servidor.
- Un solo parámetro, category, concentra varias fallas (XSS, CRLF y posible inyección SQL), lo que muestra la importancia de validar toda entrada en el servidor.
- Varias alertas de bajo riesgo, como la falta de CSP, HSTS o HttpOnly, no son graves por sí solas, pero agravan el impacto de las de riesgo alto.

¿Qué fue lo más retador?

- Distinguir entre hallazgos confirmados y posibles falsos positivos: SQL Injection y CSRF tienen confianza baja en ZAP y requieren validación manual.
- Interpretar y priorizar un volumen alto de alertas (24 en total), separando las informativas de las que realmente implican riesgo.
- Entender el contexto de cada alerta, por ejemplo, que las cookies AWSALB y AWSALBCORS pertenecen al balanceador de carga y su corrección se hace en la infraestructura. Se resolvió leyendo el detalle de cada alerta (descripción, solución y referencias) y relacionando los hallazgos entre sí.

¿Qué revisaría a continuación?

- Validar manualmente SQL Injection y CSRF, por ejemplo, con Burp Suite o pruebas dirigidas, para confirmar si son explotables.
- Analizar las alertas informativas que no se documentaron aquí, como Session Management, Cookie Poisoning y User Controllable HTML Element Attribute, además de Information Disclosure – Debug Error Messages.
- Revisar autenticación y gestión de sesiones, y ampliar la cobertura a otras secciones y endpoints de la aplicación (por ejemplo, la API REST).
- Repetir el escaneo después de aplicar las correcciones para verificar que cada hallazgo quedó resuelto.

Tabla final de hallazgos

Nivel de riesgo	Nombre	URL	Parámetro	Evidencia
Alto	Cross Site Scripting (Reflected)	/catalog	category	<script>alert(1);</script>
Alto	SQL Injection	/catalog	category	category=' → HTTP 500
Alto	Vulnerable JS Library	/resources/js/angular_1-7-7.js	-	AngularJS v1.7.7
Medio	Absence of Anti-CSRF Tokens	/catalog/product?productId=5	-	Formulario POST sin token
Medio	CRLF Injection	/catalog	category	Set-cookie: Tamper=15a29bb0…
Medio	Content Security Policy (CSP) Header Not Set	/blog	-	Missing CSP header
Medio	Missing Anti-clickjacking Header	/catalog	x-frame-options	Missing X-Frame-Options
Bajo	Application Error Disclosure	/catalog	-	HTTP/1.1 500 Internal Server Error
Bajo	Cookie No HttpOnly Flag	/blog	AWSALB	Set-Cookie sin HttpOnly
Bajo	Cookie Without Secure Flag	/blog	AWSALB	Set-Cookie sin Secure
Bajo	Cookie with SameSite Attribute None	/blog	AWSALBCORS	SameSite=None
Bajo	Cookie without SameSite Attribute	/blog	AWSALB	Sin atributo SameSite
Bajo	Strict-Transport-Security Header Not Set	/blog	-	Missing HSTS header
Bajo	X-Content-Type-Options Header Missing	/blog	x-content-type-options	Missing nosniff

Plan de remediación

Se propone atender los hallazgos según su prioridad:

N.º	Hallazgo	Riesgo	Acción principal
01	Cross Site Scripting (Reflected)	Alto	Codificar la salida y validar con lista blanca
02	SQL Injection	Alto	Consultas parametrizadas y menor privilegio
03	Vulnerable JS Library	Alto	Actualizar o reemplazar AngularJS
04	Absence of Anti-CSRF Tokens	Medio	Tokens anti-CSRF y cookies SameSite
05	CRLF Injection	Medio	Filtrar CR/LF en parámetros
06	Content Security Policy (CSP) Header Not Set	Medio	Configurar cabecera CSP
07	Missing Anti-clickjacking Header	Medio	Configurar X-Frame-Options o frame-ancestors
08	Application Error Disclosure	Bajo	Páginas de error personalizadas
09	Cookie No HttpOnly Flag	Bajo	Activar HttpOnly en cookies
10	Cookie Without Secure Flag	Bajo	Activar Secure en cookies
11	Cookie with SameSite Attribute None	Bajo	SameSite Lax/Strict
12	Cookie without SameSite Attribute	Bajo	SameSite Lax/Strict
13	Strict-Transport-Security Header Not Set	Bajo	Configurar HSTS
14	X-Content-Type-Options Header Missing	Bajo	Enviar nosniff

Orden sugerido de atención

- Inmediata: hallazgos de riesgo alto (XSS, SQL Injection, librería vulnerable).
- Corto plazo: hallazgos de riesgo medio (CSRF, CRLF, CSP, clickjacking).
- Planificada: hallazgos de riesgo bajo (errores, cookies, HSTS, nosniff), que pueden resolverse en gran parte con cambios de configuración del servidor y el balanceador.
- Después de aplicar las correcciones, repetir el escaneo con ZAP y validar manualmente los hallazgos de menor confianza, como SQL Injection y CSRF.